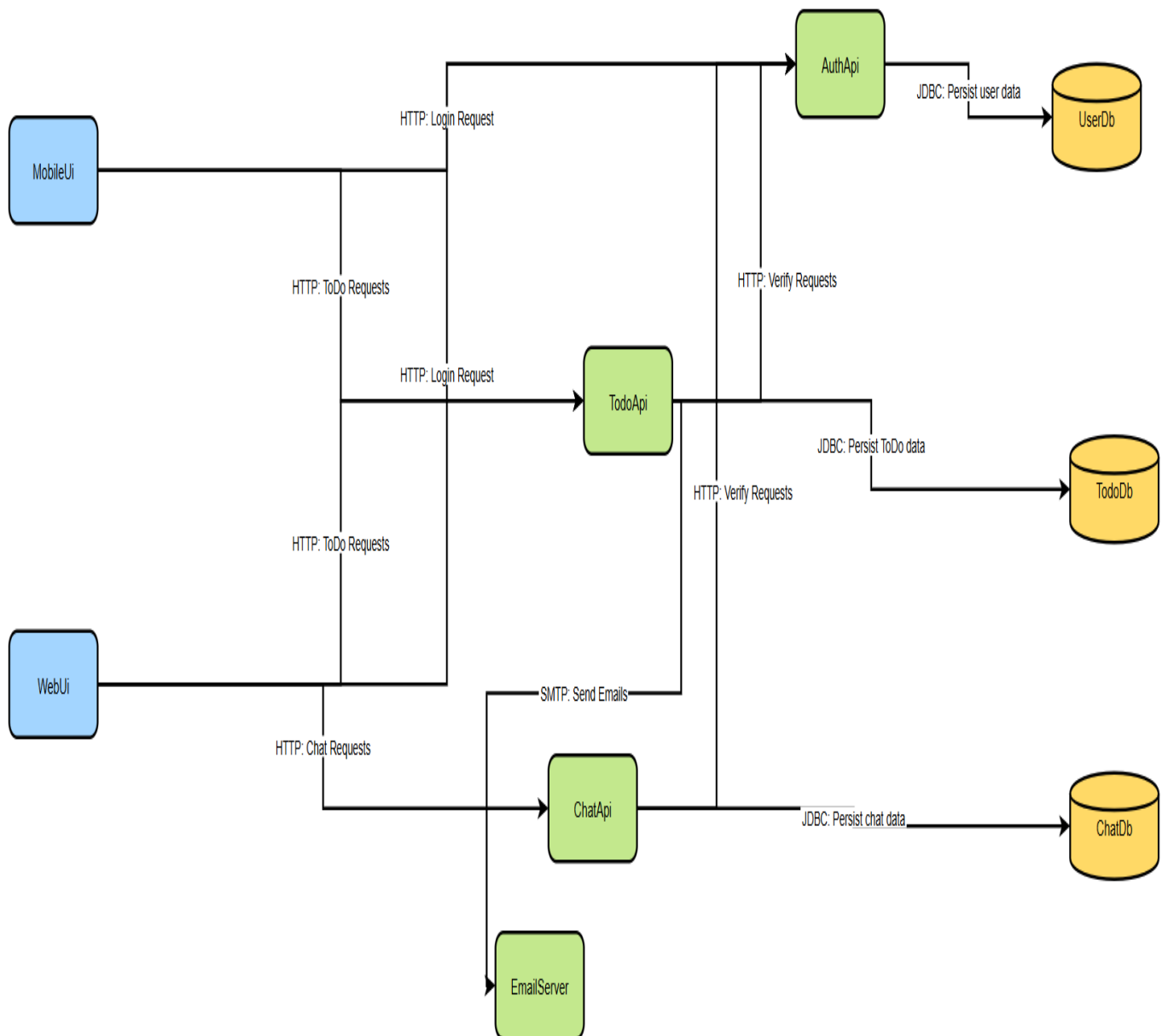


Aufgabe 1



Aufgabe 2.1

a)

```
CREATE TABLE IF NOT EXISTS todos (  
    id INT PRIMARY KEY,  
    titel VARCHAR(255) NOT NULL,  
    description TEXT  
);
```

b)

```
INSERT INTO todos (id, titel, description)  
VALUES (1, 'Dekorieren', 'Es ist nun endlich so weit! Mit dem 01. November  
wird es Zeit, zügig die Weihnachtsdekorationen auspacken.');
```

c)

```
SELECT description  
FROM todos  
WHERE description LIKE '%Weihnacht%';
```

Ergebnisse:

1. Es ist nun endlich so weit! Mit dem 01. November wird es Zeit, zügig die Weihnachtsdekorationen auspacken.
2. Bald sollte ich Weihnachtsplaetzchen backen.

Aufgabe 2.2

a) EntwickLUnGPrOgrAMMII

```
public static void main(String[] args) {  
    int[] arrayIndexes = {  
        20, 44, 50, 13, 17, 33, 41,  
        68, 77, 44, 29, 72, 48, 71,  
        37, 48, 11, 69, 5, 65, 65  
    };  
  
    try {  
        JdbcConnectionSource connectionSource = new JdbcConnectionSource(DATABASE_URL, USERNAME, PASSWORD);  
        Dao<Letter, Integer> letterDao = DaoManager.createDao(connectionSource, Letter.class);  
  
        // Find and print the word  
        StringBuilder word = new StringBuilder();  
        for (int id : arrayIndexes) {  
            Letter letter = letterDao.queryForId(id);  
            if (letter != null) {  
                word.append(letter.getLetter());  
            } else {  
                System.out.println("Kein Buchstabe für ID: " + id);  
            }  
        }  
        System.out.println("Das Lösungswort ist: " + word.toString());  
    }  
}
```

b) ID's für V : 52 , 78

ID's für b : 9, 32 , 58

ID's für t : 50 , 76

```
private static void findIdsForLetter(Dao<Letter, Integer> letterDao, String letter) throws SQLException {  
    List<Letter> letters = letterDao.queryForEq("letter", letter);  
    System.out.println("IDs for letter '" + letter + "':");  
    for (Letter l : letters) {  
        System.out.println(l.getId());  
    }  
}
```

```
// Find IDs for specific letters  
findIdsForLetter(letterDao, letter: "V");  
findIdsForLetter(letterDao, letter: "b");  
findIdsForLetter(letterDao, letter: "t");
```

c) Summe der ID's : 4167

Durchschnittswert : 50,817

```
// Sum and average of all IDs  
List<Letter> letters = letterDao.queryForAll();  
int sum = 0;  
for (Letter letter : letters) {  
    sum += letter.getId();  
}  
double average = letters.isEmpty() ? 0 : (double) sum / letters.size();  
System.out.println("Summe der IDs: " + sum);  
System.out.println("Durchschnittswert der IDs: " + average);  
  
connectionSource.close();  
catch (SQLException e) {  
    e.printStackTrace();  
catch (Exception e) {  
    throw new RuntimeException(e);
```

Aufgabe 3

a)

```
{ "categories": [ "history" ],
  "created_at": "2020-01-05 13:42:19.324003",
  "icon_url": "https://api.chucknorris.io/img/avatar/chuck-norris.png",
  "id": "3qr6hlt6rouy2t2jtlNzzw",
  "updated_at": "2020-01-05 13:42:19.324003",
  "url": "https://api.chucknorris.io/jokes/3qr6hlt6rouy2t2jtlNzzw",
  "value": "The Great Wall of China was originally created to keep Chuck Norris out. It failed miserably." }
```

b)

```
{
  "args": {},
  "data": {
    "key": "pe2ws23",
    "purpose": "This is a test."
  },
  "files": {},
  "form": {},
  "headers": {
    "host": "postman-echo.com",
    "x-request-start": "t=1730906282.693",
    "connection": "close",
    "content-length": "59",
    "x-forwarded-proto": "https",
    "x-forwarded-port": "443",
    "x-amzn-trace-id": "Root=1-672b88aa-24c96b9614cac7d33bb55cac",
    "content-type": "application/json",
    "user-agent": "PostmanRuntime/7.42.0",
    "accept": "*/*",
    "cache-control": "no-cache",
    "postman-token": "b0f9c2b2-f1f9-4ef0-b448-9f80b86aa23c",
    "accept-encoding": "gzip, deflate, br",
    "cookie": "sails.sid=s%3AhzXAyU-G3dKfwcojUp1WSPTucRUS4Xhb.ps84Jcim88pFoSrKFCgcQkwYRoTi3bFT3FCAnuU7p%2Bg"
  },
  "json": {
    "key": "pe2ws23",
    "purpose": "This is a test."
  },
  "url": https://postman-echo.com/post }
```

c)

1. **Abrufen aller DVDs mit allen Filtermöglichkeiten**

- **Methode:** GET
- **Pfad:**
`/dvds?category=$category&titleContains=$titleString&isRestricted=$isRestricted`
- **Beschreibung:** Liefert eine Liste aller verfügbaren DVDs mit optionalen Filtern: Kategorie (`category`), Wort im DVD-Titel (`titleContains`) und Altersbeschränkung (`isRestricted`), wobei `isRestricted=true` nur DVDs ab 18 zurückgibt.

2. **Abrufen einer einzelnen DVD über ihre ID**

- **Methode:** GET
- **Pfad:** `/dvds/$id`
- **Beschreibung:** Liefert die Details einer einzelnen DVD, welche über ihre ID identifiziert werden.

3. **Erstellen einer neuen DVD**

- **Methode:** POST
- **Pfad:** `/dvds`
- **Beschreibung:** Erstellt eine neue DVD-Einheit mit den im Request-Body bereitgestellten Attributen.

4. **Aktualisieren einer bestehenden DVD über ihre ID**

- **Methode:** PUT
- **Pfad:** `/dvds/$id`
- **Beschreibung:** Aktualisiert die Details einer bestehenden DVD, welche über ihre ID klassifiziert wird.

5. **Löschen einer DVD über ihre ID**

- **Methode:** DELETE
- **Pfad:** `/dvds/$id`
- **Beschreibung:** Löscht eine DVD, welche über ihre ID festgestellt wird.